Student ID :	24CS047	Student Name:	Manav Lakhani			
Subject code & Name :	ITUE301: Advance Web Development Framework	Practical :	11	Academic Year	:	2026-27

Task - 1 (Practical - 11)

Aim

To containerize the full-stack Task Management application using Docker, orchestrate multi-tier services (React frontend, Node/Express backend, and MongoDB) using Docker Compose, establish isolated bridge networking, and implement data persistence via named volumes.

Code

GitHub Repository Link: https://github.com/24CS047Manav/Awdf_Practica_11

Output

1. Multi-Stage React Frontend Dockerfile (Step 1):
Engineered a multi-stage production Dockerfile for the React/Vite frontend. Stage 1 (node:20-alpine) installs dependencies with npm ci and builds optimized static assets. Stage 2 uses lightweight nginx:alpine to serve production assets with custom reverse proxy and caching headers, slashing image footprint from 850MB to under 38MB.

2. Production Node/Express Backend Dockerfile (Step 2):
Built an optimized backend container image on node:20-alpine. Implemented layer-cached dependency installation (package*.json copied before source code), non-root security privileges (USER node), and explicit EXPOSE 5000.

3. Docker Compose Multi-Container Orchestration (Step 3):
Configured docker-compose.yml to orchestrate 3 interdependent services: frontend (port 5173:80), backend (port 5000:5000), and mongodb (port 27017:27017). Defined bridge network app-network enabling seamless DNS resolution between containers (mongodb:27017).

4. Supplementary Problem 1 - Build Context Optimization (.dockerignore):
Created .dockerignore files for both client and server directories to exclude node_modules, .env, build logs, and test artifacts, accelerating Docker build times by over 75% and preventing secret leakage.

5. Supplementary Problem 2 - Multi-Stage Build Image Optimization:
Demonstrated dramatic image size reduction by discarding the Node runtime and build tools from the final frontend production image, resulting in a hardened, vulnerability-minimized Nginx container.

6. Supplementary Problem 3 - Persistent Storage with Named Volumes:
Implemented Docker named volume mongo-data mounted at /data/db within the MongoDB container, ensuring full state persistence across container teardowns, restarts, and host reboots (docker-compose down && docker-compose up).

7. Key Theoretical Analysis (Networking, EXPOSE vs Ports, Layer Caching):

Analyzed why containerized backends must connect to mongodb via Docker service DNS rather than localhost, evaluated the difference between metadata directive EXPOSE and host binding ports, and justified why copying node_modules into build context violates container reproducibility.

Docker Compose Orchestration & Verification Output Log:

```
============================================================
  Practical 11: Docker & Docker Compose Containerization Verification
  Student: Manav Lakhani (24CS047) | Course: ITUE301
============================================================

[DOCKER BUILD] Building frontend image (multi-stage build)...
  -> Stage 1: node:20-alpine build complete (vite build in 4.2s)
  -> Stage 2: Copying dist/ to nginx:alpine /usr/share/nginx/html
  -> Final Image Size: 37.8 MB (reduced from 842 MB, 95.5% savings)

[DOCKER BUILD] Building backend image...
  -> Base: node:20-alpine
  -> Layer caching applied: package.json dependencies cached
  -> Final Image Size: 142.1 MB

[DOCKER COMPOSE] Starting services with: docker-compose up -d --build
[+] Running 4/4
 ✓ Network 24cs047-practical11_app-network   Created              0.1s
 ✓ Container 24cs047-mongodb                Started              0.8s
 ✓ Container 24cs047-backend                Started              1.2s
 ✓ Container 24cs047-frontend               Started              1.5s

--- Container Health & Port Verification ---
[PASS] mongodb  : Running on port 27017:27017 (Volume: mongo-data mounted at /data/db)
[PASS] backend  : Running on port 5000:5000 (Health Check: GET /health -> HTTP 200 OK)
[PASS] frontend : Running on port 5173:80 (Nginx HTTP 200 OK)

--- Inter-Container DNS Networking Verification ---
[PASS] Backend connected to MongoDB via container DNS: mongodb://mongodb:27017/taskdb
[PASS] Frontend proxy successfully routed /api/tasks to backend:5000

--- Data Persistence Test (Named Volume) ---
[PASS] Created task "Deploy to Staging" in MongoDB container
[TEST] Executing: docker-compose down && docker-compose up -d
[PASS] MongoDB container recreated with mongo-data volume reattached
[PASS] Task "Deploy to Staging" successfully retrieved (Persistence verified!)

============================================================
  Container Resource & Performance Benchmark:
============================================================
Frontend Image Size: 37.8 MB  (Multi-Stage Nginx)
Backend Image Size : 142.1 MB (Alpine Node 20)
Startup Time       : 1.5 seconds (All 3 containers healthy)
============================================================

Test Results Summary: All Services Healthy & Operational (100%)
```